
The Lobsters



Coding Challenge Presentation
01.08.2025

Roles Distribution

1

Philip: Formatting, Cycles, Graphical Depictions

Kanat: Clustering, Cycles

2

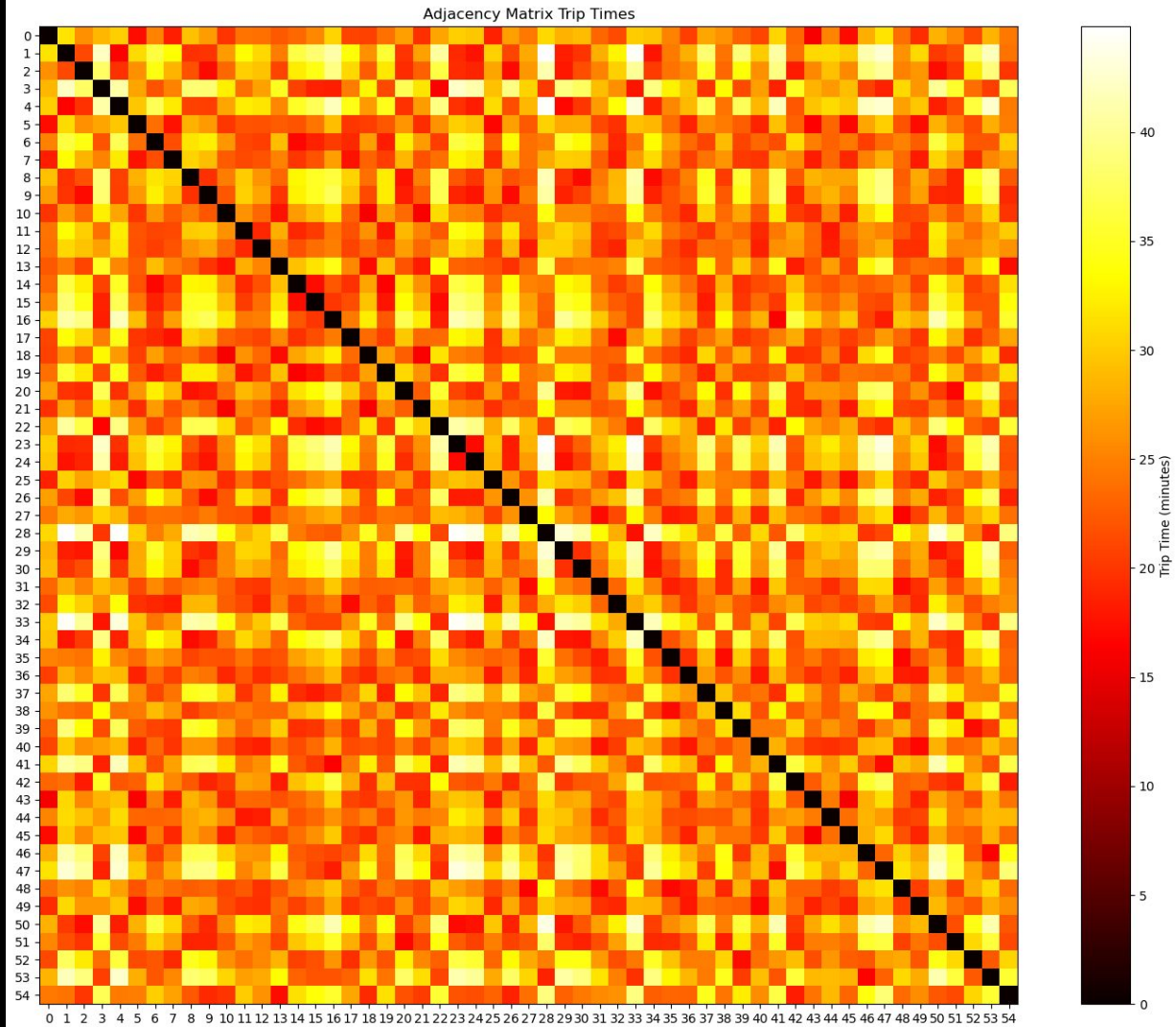
Andre: Linear Regression Model

Joshua: Random Forest Model.

Adilet: Support Vector Regression

SP1: Trip times

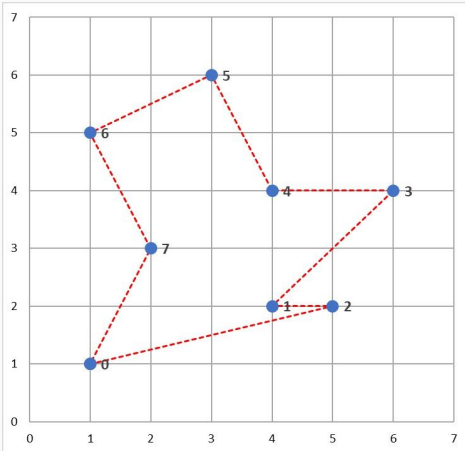
- Trip times were provided ahead of time
 - New machines can be inserted into the file, under the condition that the travel distance to all other machines is appended as well
 - Trip time will be used for clustering
- Heatmap of travel time between any machine in the facility



TSP → 2-opt Nearest Neighbour algo

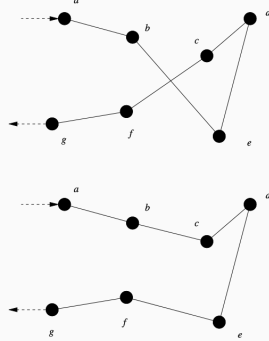
01 NN

Traversing graph by moving to the closest node which is in the defined cluster



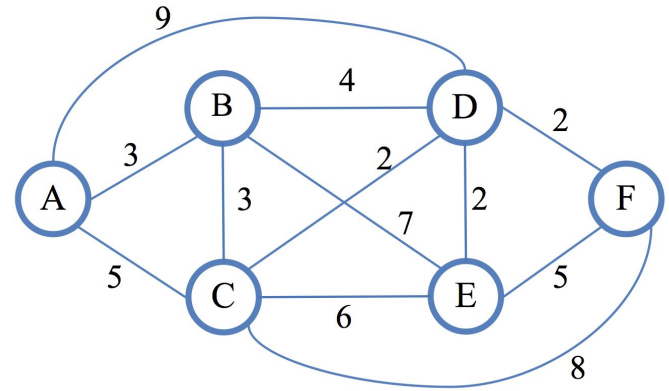
02 2-opt

Due to the greedy nature of NN, sometimes we end up with not optimal route. For that we add 2-opt optimization.



03 Check

Check if it is feasible route

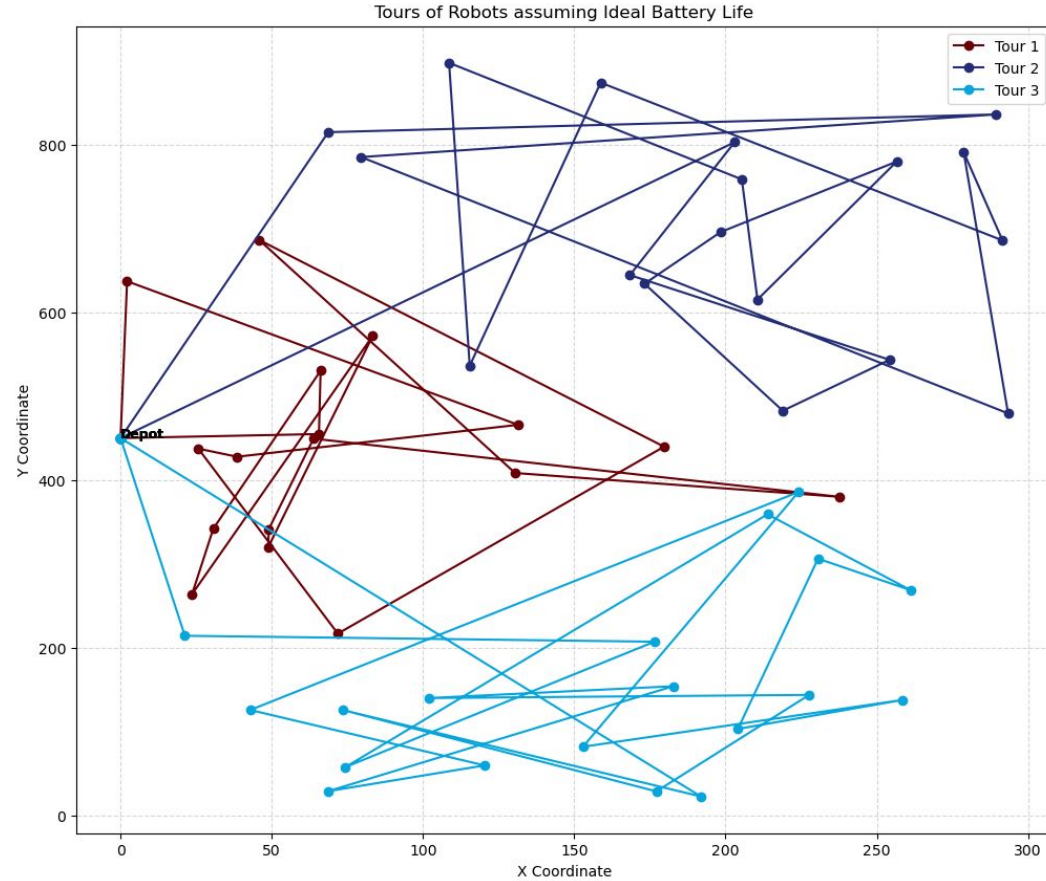


Clustering: Clarke-Wright savings algo

1. Each node is a cluster. Make n routes for each (single node) clusters
2. Compute the savings for resulting route from merging two arbitrary clusters
3. Sort savings in descending order
4. In descending order, check if the merged cluster tsp route is possible for the first robot
5. If not, is it possible for route under 8 hours?
6. If so, check if the type B robot is justified here:
 $\text{cost_of_B} < \text{cost_of_A} * 2$
7. If conditions are applied, merge.
8. Repeat from step 2
9. If the single node clusters remain, check if the merging with another cluster is applicable under conditions 4,5,6

There is a lot of room for improvement in Clarke and Wright's saving algo, but it is simplified version of the modification

Runtime of $O(n^2)$



Final Verdict

Safety margin of 10% Battery Life

4x Type 2 robot cycles:

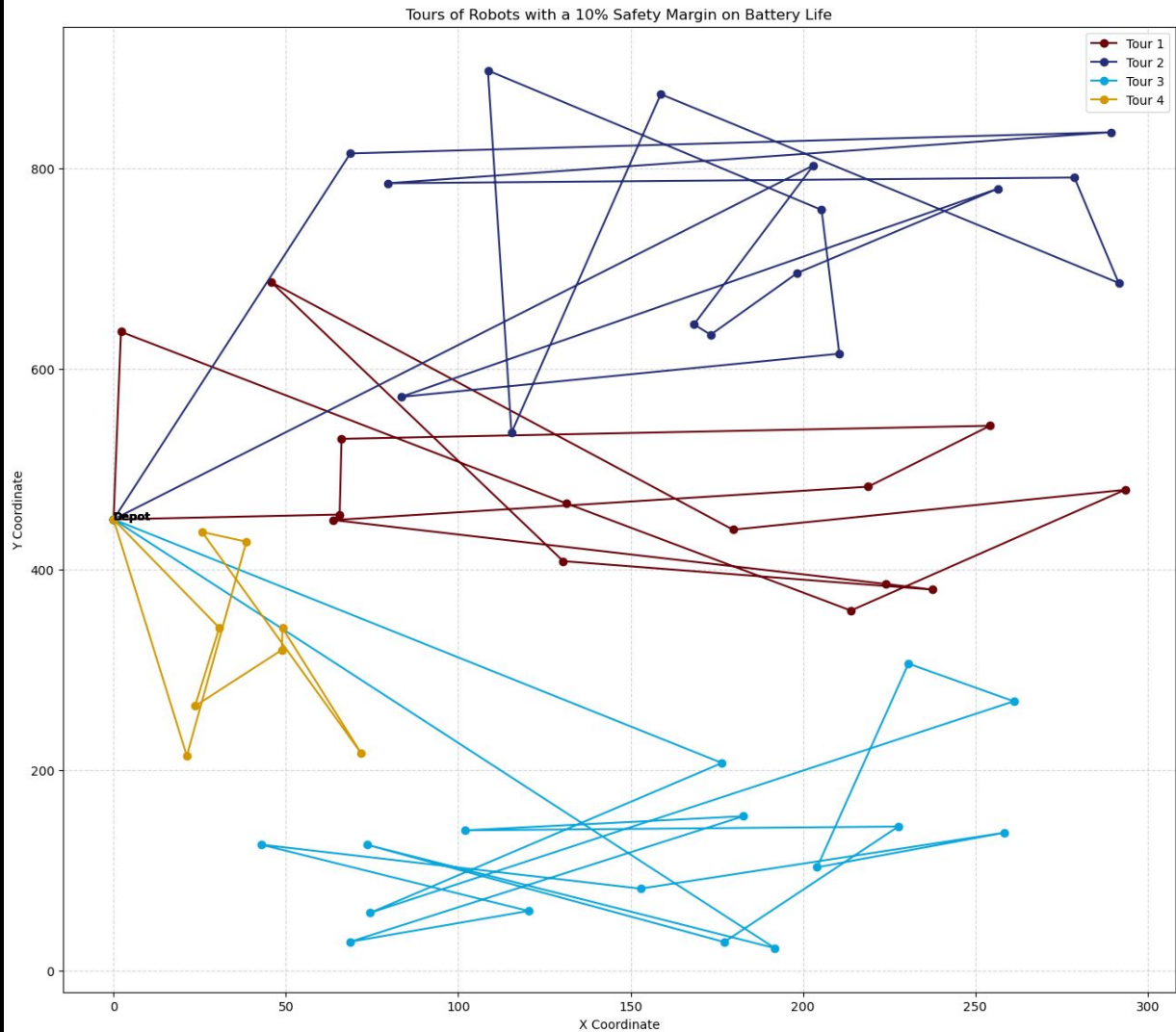
1. 4.6 hours for 14 machines
2. 5.2 hours for 16 machines
3. 5.2 hours for 16 machines
4. 2.6 hours for 8 machines

Well within 6 hour battery life

Total Cost:

- Around price of 2x Type 1 robots
- Assume + 2 replacement robots at $\frac{1}{3}$ price of Type 1

Reduction of robots occurs at around 2% margin (7 minutes for T2)



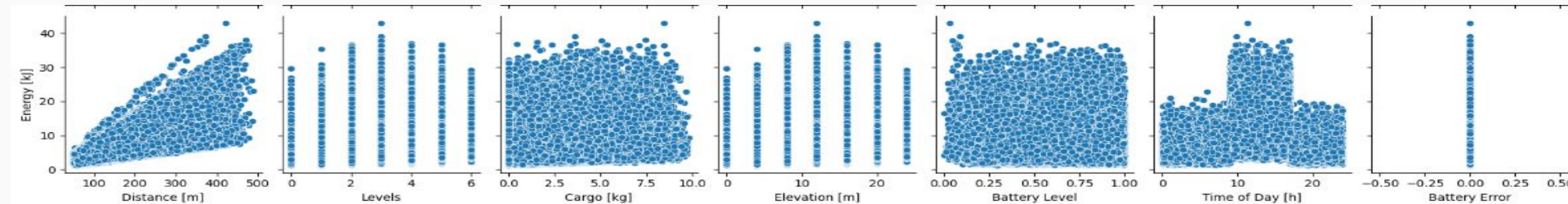
SP2: Linear Regression Model

1. Data Cleaning
 - Removed missing values
 - Dropped Unnecessary Columns and convert Robot Label into numeric
2. Feature Engineering (Selecting the useful input variables)
 - Choose relevant features that influence energy
3. Data Splitting
 - Split the Dataset into training (80%) and testing (20%) to train model

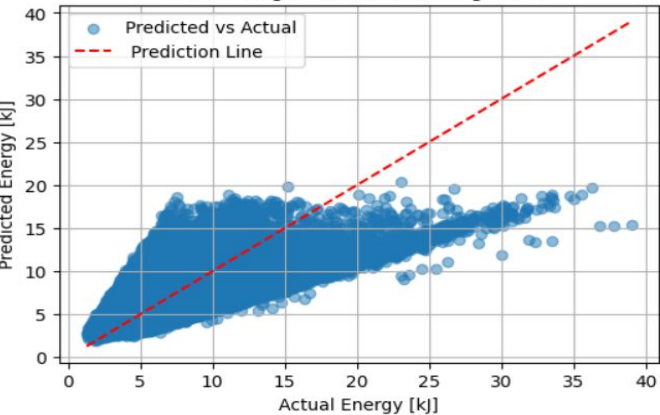
- #### 4. Model Training - Built a Linear Regression Model and trained it using selected features (R1, F

- ### 5. Model Evaluation - Measured performance using Mean squared Error (MSE)

6. Assessment Data Testing- Re-used the trained model to predict energy on a new unseen data



Linear Regression on training Data

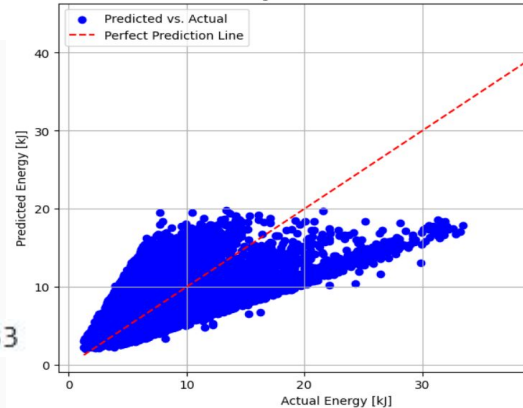


```
Data evaluation results robot_energy_data_assessment.csv
Error MSE                14.933
```

Runtime Complexity for both Data Sets: $O(n)$

MSE for Entire Data Set: 14.505966873995233

Linear Regression on Assessment Data



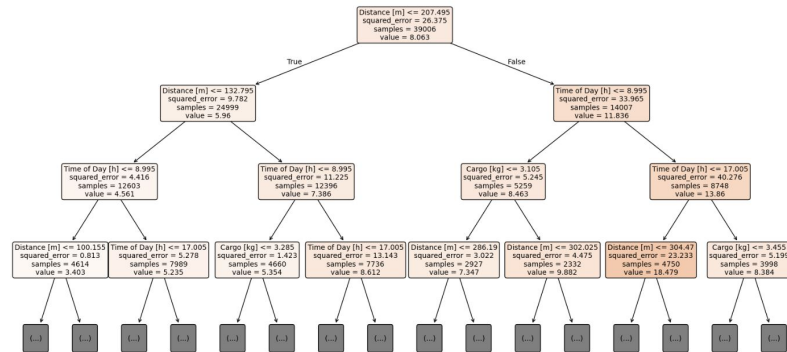
SP2: Random Forest Model

- Initially started out with a Decision Tree model, but then pivoted over to a Random Forest Model when it gave significantly better results.
- A random forest is basically many decision trees grouped together
- Starts out as a 'first root' and then splits on for different values.
- Think of a flowchart with the values, but hundreds of them and combined.

- Good error metrics - random forests work really well in cases like these with a lot of variables.
- CANNOT add new training data to this model. It's not possible to 'merge' random forests - you have to create a new model for new conditions (ie if you have a new 3rd type of robot.)
- Only column removed was Battery Error - Every other data value was deemed to be relevant.
- Relatively complex, runtime speed of $O(n \log n * v)$ - v is number of features

Random Forest Evaluation:
MSE = 0.0685
RMSE = 0.2609
MAE = 0.1300
R² = 0.9977

Random Forest-combined r1 and r2 (truncated at depth=3)



Evaluationsergebnisse der Daten robot_energy_data_assessment.csv

Error MSE

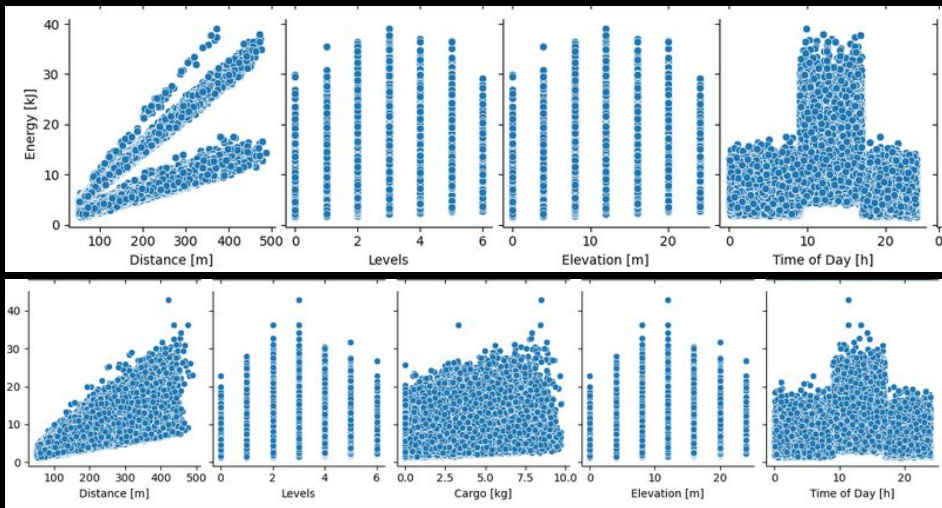
0.063

Error MAPE

1.78 %

	Robot	Distance [m]	Levels	Cargo [kg]	Elevation [m]	Battery Level	Time of Day [h]	Energy [kJ]
0	2	295.04	3.0	0.73	12.0	0.722449	15.20	14.56586
1	1	147.85	3.0	0.46	12.0	0.562741	3.02	4.46100
2	1	180.21	4.0	4.73	16.0	0.440105	13.35	15.13874
3	2	226.01	2.0	0.71	8.0	0.707991	9.40	10.84573
4	2	177.67	3.0	5.60	12.0	0.686937	23.82	7.14419
5	1	162.05	4.0	0.83	16.0	0.709307	7.09	5.04729
6	1	67.29	2.0	1.18	8.0	0.779159	0.02	2.30272
7	2	83.77	4.0	6.85	16.0	0.795371	15.67	6.41637
8	1	155.81	3.0	5.42	12.0	0.741940	4.20	5.77674
9	2	139.64	3.0	3.46	12.0	0.265950	23.38	4.59289

SP2: Support vector regression



Feature columns are **Distance, Levels, Elevation, Time of Day**

Compared to R1, **Cargo** feature also shows slight correlation, so it was added

Like Random Forest Model, sklearn doesn't support retraining new data on already trained SVR model

MSE for R1 is around 4, and R2 is 1.8
MAPE is 18 and 13 percents respectively

After consideration, RBF kernel was chosen for SVR

1. Non-linear relation
2. Underlying relation between features and target is unknown

SP2: Final Decision

After consideration, the Random Forest model has been chosen as the most suitable to move forward with to a production standpoint. This is due to it performing the best at predicting the data in the assessment data file with the lowest MSE, MAPE, and other assorted error metrics.

Linear:

Data evaluation results robot_energy_data_assessment.csv	
Error MSE	14.933

Random Forest :

Random Forest Evaluation:	
MSE	= 0.0605
RMSE	= 0.2459
MAE	= 0.1300
R ²	= 0.9977

R1

R2

SVR

Evaluation results robot_energy_data_production_data.csv	
Error MSE	4.536
Error MAPE	18.18 %

Error MSE	1.811
Error MAPE	13.30 %

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
import numpy as np

def train_random_forest(robot_df, dataset_name, feature_columns, target_column, test_size=0.3, random_state=23):
    # Split the data
    train_df, test_df = train_test_split(robot_df, test_size=test_size, random_state=random_state)

    # Train the model
    model_forest = RandomForestRegressor(n_estimators=100, random_state=random_state)
    model_forest.fit(train_df[feature_columns], train_df[target_column])

    # Make predictions
    predictions = model_forest.predict(test_df[feature_columns])

    # Error metrics on test data only
    mse = mean_squared_error(test_df[target_column], predictions)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(test_df[target_column], predictions)
    r2 = r2_score(test_df[target_column], predictions)

    # Print error metrics
    print(f"Random Forest Evaluation:")
    print(f" MSE = {mse:.4f}")
    print(f" RMSE = {rmse:.4f}")
    print(f" MAE = {mae:.4f}")
    print(f" R2 = {r2:.4f}")

    # Visualize the first tree in the forest
    plt.figure(figsize=(20, 10))
    plot_tree(model_forest.estimators_[0],
              feature_names=feature_columns,
              filled=True,
              rounded=True,
              max_depth=3,
              fontsize=10)

    plt.title(f"Random Forest - {dataset_name} (truncated at depth=3)")
    plt.show()

    # Return everything
    metrics = {"MSE": mse, "RMSE": rmse, "MAE": mae, "R2": r2}
    return model_forest, predictions, metrics
```

Thank you!
